WEB 250 asgn08 assignment code flow

Student review handout 4 of 5 | Guide for your independent work

This guide follows the asgn08 requirements from the first request for public/index.php, through the directory, to one bird's details. Use it to explain your own code and predict the next method call. The completed model has five methods, and each method has a specific job.

1 Start a new request at the index

The browser requests public/index.php. If you arrive through your course index, clicking its asgn08 link starts this request. PHP begins the requested file; it does not execute every PHP file in the folder.

The reference solution first gives its display variables starting values. Then the index includes private/initialize.php inside its existing try block. The steps below happen before any summary counts are displayed.

Order	Statement or file	What happens next
1	initialize.php loads functions.php	The output helper h() becomes available. Its body runs only when called.
2	Load db_credentials.php	The existing DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASS constants receive their local settings.
3	Load Database.php and Bird.php	PHP knows these classes. Loading a class definition does not call all its methods.
4	$pdo = Database::connect();	Enter the connection method. On this request's first call, build the DSN and create one PDO object. Return that object to $pdo.
5	Bird::set_database($pdo);	Store that same object in the Bird class's private static $database property. No birds are fetched here.
6	Initialization reaches its end	Resume index.php immediately after its require_once statement.

Inside Database::connect(), the saved connection starts as null for a new request. The first call reaches new PDO(...). A later call within that same request returns the already saved object. Bird::set_database() makes the connection available to the model; it does not open a second connection.

Your original asgn07 code may have used different variable names or opened the connection inside the index. Match the responsibilities and order. For completed asgn08, the assignment requires the shared initialization to supply the connection and call Bird::set_database() after it exists.

2 Return an integer for the index summary

The index retains its SELECT DATABASE() query and the asgn07 image and image-link counts. The bird count now follows this method call:

```
$bird_count = Bird::count_all();
```

Inside count_all(), the model runs SELECT COUNT(*) FROM birds. fetchColumn() reads the count from the result, and the integer cast produces the required return type. The returned integer is assigned to the index's $bird_count.

This path does not call find_all() or from_row(). Counting records does not require making an object for every row. The index puts its counts into the existing summary table and sends HTML back to the browser. With the original seed, the three counts are 8 birds, 17 images, and 18 links; your added data may change them.

3 Follow a link to the directory

Click **Bird directory**. The browser now requests public/birds.php. This is a new request with its own initialization and connection. The previous index request has finished.

After initialization, the directory asks for its data:

```
$birds = Bird::find_all();
```

The page waits while the method performs these steps:

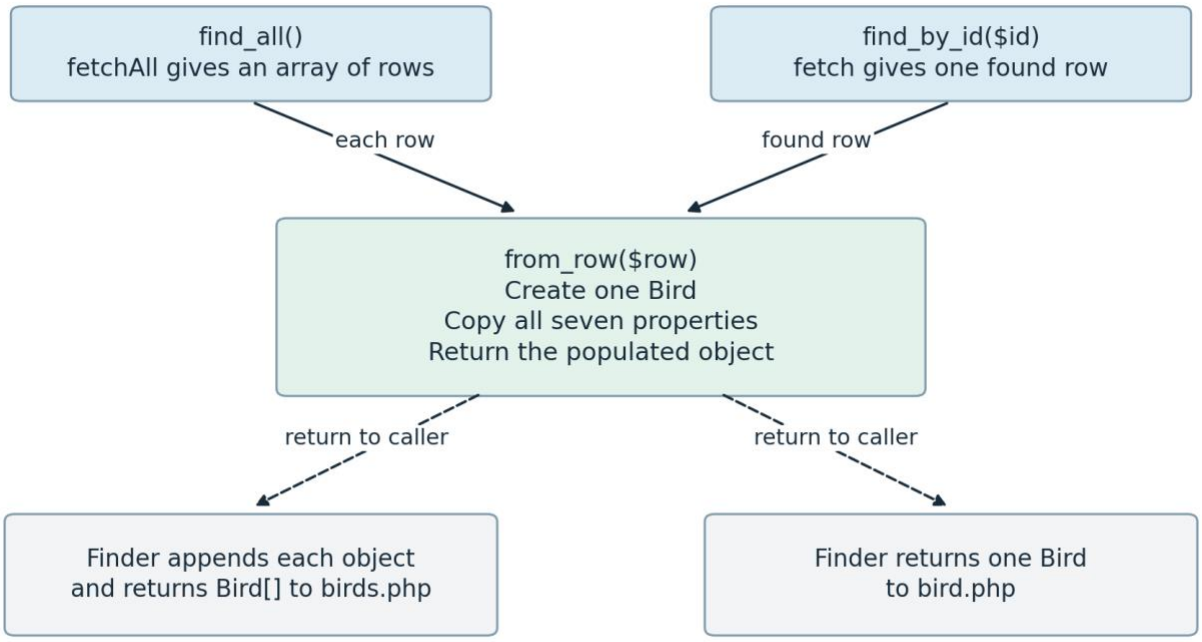
1. Run a fixed SELECT for all seven bird columns, ordered by common name.

2. Use fetchAll(PDO::FETCH_ASSOC) to obtain an array of associative rows.

3. Create a local empty $birds array.

4. For each $row, call self::from_row($row) and append the returned object.

5. Return the completed array to the page's pending assignment.



How the two finder methods share the row mapper

The directory then loops over its returned objects. It prints the common name, scientific name, habitat, and season, and builds a details link using each object's actual $id. The array position in $birds is not the record ID.

If no rows match, fetchAll() gives an empty array. The loop runs zero times, find_all() returns [], and the page shows its no-records message. A successful empty result is different from a database exception.

4 Pause inside the shared row mapper

from_row(array $row) receives one associative array. It creates one Bird with new self(), copies the seven columns into that object's properties, and returns the object.

Before mapping	After mapping
$row['id']	$bird->id, stored as an integer
$row['common_name']	$bird->common_name
$row['scientific_name']	$bird->scientific_name
$row['family_name']	$bird->family_name
$row['primary_habitat']	$bird->primary_habitat
$row['seasonal_status']	$bird->seasonal_status
$row['description']	$bird->description

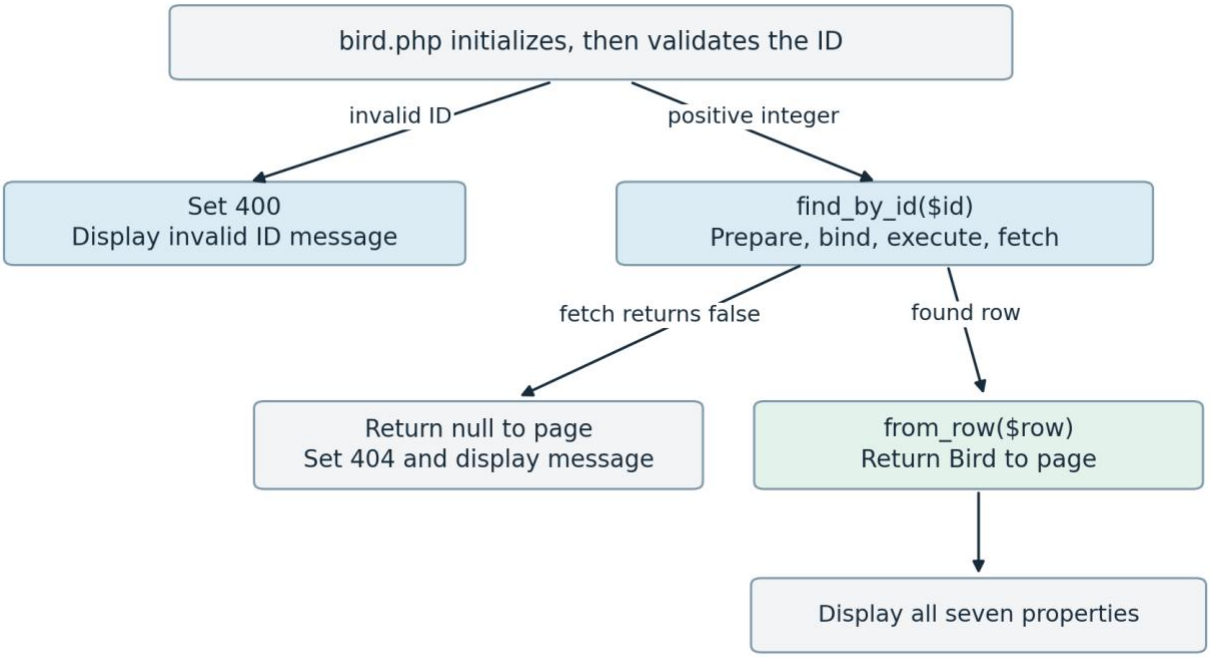
The helper does not query MySQL. Its caller already fetched the row. The helper is private because the class handles this internal conversion. Both finder methods use it, so there is only one block of property assignments to maintain.

In the supplied starter, self as a return type means a Bird here, and ?self permits a Bird or null. The new self() expression constructs an instance of this class. Creating the instance alone gives default property values; the following assignments fill them.

5 Follow a details link

Click a details link generated by the directory. For example, the URL ends in bird.php?id=2 if that selected record's ID is 2. Use the ID actually present in your data.

The reference solution's detail page loads initialization first, then uses filter_input() with FILTER_VALIDATE_INT and a minimum of 1. It checks the validation result before calling the finder.



Detail request branches for invalid missing and found records

For a valid positive ID, the page calls Bird::find_by_id($id). In this method:

1. The SQL contains WHERE id = :id.
2. prepare() creates the statement for that SQL.
3. bindValue(':id', $id, PDO::PARAM_INT) associates the integer with the placeholder.
4. execute() executes the statement with that value.
5. fetch(PDO::FETCH_ASSOC) retrieves one row, or returns false when no row is found.
6. If there is no row, the method returns null. Otherwise, it calls from_row() and returns that helper's Bird object.

The SQL text can still contain :id in the debugger after binding. Binding does not require PHP to replace the placeholder with a number in the visible SQL string.

Back in the page, null produces the friendly 404 response. A returned Bird supplies all seven displayed properties. Missing, noninteger, zero, and negative IDs take the friendly 400 path and never call find_by_id(). In this reference version, initialization has already run even on that invalid-ID path.

Set the response code before sending the HTML. The page's h() helper escapes database text as it is inserted into HTML. It does not change the stored data or perform the parameter binding.

6 Keep the return values straight

Method	Caller expects	Does it create a Bird object
set_database()	No return value	No
count_all()	One integer	No
find_all()	An array of objects, possibly []	One per returned row, through the helper
find_by_id()	One object or null	One if found, through the helper
from_row()	One object	Yes

While from_row() is running, the Call Stack identifies the caller as either find_all() or find_by_id(). A variable named $bird in this helper is local to that call. Its returned object reaches the public page through the finder. Using the same variable name in different places does not make those variables global.

Check your understanding: Explain why a directory request calls from_row() several times, a successful detail request calls it once, and an index count calls it zero times. Then use handout 5 to verify your explanation in the debugger.

Code reference: asgn08 Active Record Directory instructions Sections 5.1 through 5.4 and the accompanying minimal reference solution. This guide describes successful local setup with the existing wnc_birds database.

PHP references: PDOStatement fetch, fetchAll, fetchColumn, bindValue, and filter_input.